

Tareas

US-C2 · Scheduler de publicaciones [8 SP · Must]

Como sistema, quiero un scheduler que dispare las publicaciones al llegar su hora, para ejecutar sin intervención manual.

Criterios de aceptación:

- Ejecuta el job en la hora programada con precisión razonable
- Se recupera correctamente tras un reinicio (job store persistente)

Tareas sugeridas:

- Integrar un scheduler con job store persistente (APScheduler/Quartz)
- Registrar y cancelar jobs de publicación
- Recuperación de jobs tras reinicio
- Pruebas de disparo y recuperación

US-C5 · Manejo de cuotas y errores [5 SP · Must]

Como sistema, quiero reintentar ante errores transitorios/de cuota y marcar los fallos definitivos, para dar robustez a la publicación.

Criterios de aceptación:

- Errores transitorios se reintentan con backoff
- Superado el límite, emite publish.failed con causa

Tareas sugeridas:

- Mapear los errores de la API (transitorios vs definitivos)
- Implementar backoff y límite de reintentos
- Emitir publish.failed con el detalle del error
- Pruebas de los escenarios de error

US-C6 · Máquina de estados de publicación [3 SP · Should]

Como sistema, quiero estados pending → publishing → published/failed, para dar trazabilidad al proceso.

Criterios de aceptación:

- Las transiciones reflejan el resultado real de la API
- GET /publish/{id}/status expone el estado actual

Tareas sugeridas:

- Definir estados y transiciones
- Implementar GET /publish/{id}/status
- Pruebas de la máquina de estados

División Tareas

US-C2 · Scheduler de publicaciones [8 SP · Must]

Como sistema, quiero un scheduler que dispare las publicaciones al llegar su hora, para ejecutar sin intervención manual.

Criterios de aceptación:

- Ejecuta el job en la hora programada con precisión razonable
- Se recupera correctamente tras un reinicio (job store persistente)

Tareas sugeridas:

- Integrar un scheduler con job store persistente (APScheduler/Quartz)
- Registrar y cancelar jobs de publicación
- Recuperación de jobs tras reinicio
- Pruebas de disparo y recuperación

Subtareas:

- **Subtarea 2.1 - Integración de librería Scheduler:** Instalar y configurar una librería de calendarización (ej. APScheduler en Python o BullMQ/node-cron en Node.js) conectándola a PostgreSQL para usar un *job store persistente*.
-
- **Subtarea 2.2 - Lógica de registro de Jobs:** Crear un servicio interno que, al momento de programar un video (conectado a la acción de US-C1), registre el *job* en el scheduler apuntando a la fecha y hora indicadas.
-
- **Subtarea 2.3 - Ejecución de la tarea (Job Handler):** Desarrollar la función que el scheduler disparará a la hora programada. Esta debe recuperar los datos de la tabla **publication**, verificar su estado y llamar al servicio **Publisher** (el cual estará usando el Mock implementado en US-C4).
-
- **Subtarea 2.4 - Estrategia de recuperación:** Configurar el scheduler para que, al reiniciar el contenedor Docker, recupere los jobs pendientes o atrasados desde PostgreSQL y los reanude sin intervención manual.
-
- **Subtarea 2.5 - Pruebas de integración:** Escribir pruebas que simulen el paso del tiempo para verificar que el job se dispara correctamente y probar el reinicio del sistema asegurando la persistencia.

US-C2 · Scheduler (comprometida, remanente re-estimado)

- 2.1–2.3: hechas en el Sprint 1; verificarlas contra PostgreSQL en Docker.
- 2.4: servicio interno para cancelar y reprogramar jobs.
- 2.5: reconciliación al arranque, que cubre el job vencido fuera del margen, la publicación atascada en **publishing** y los jobs huérfanos. Incluye sus transiciones y un ADR con la política elegida.
- 2.6: pruebas de integración en PostgreSQL: disparo dentro de la precisión que fije el PO, persistencia tras reinicio y los casos de 2.5.
- 2.7: guion y ensayo de la demo de reinicio desde un clon limpio.

US-C5 · Manejo de cuotas y errores [5 SP · Must]

Como sistema, quiero reintentar ante errores transitorios/de cuota y marcar los fallos definitivos, para dar robustez a la publicación.

Criterios de aceptación:

- Errores transitorios se reintentan con backoff
- Superado el límite, emite `publish.failed` con causa

Tareas sugeridas:

- Mapear los errores de la API (transitorios vs definitivos)
- Implementar backoff y límite de reintentos
- Emitir `publish.failed` con el detalle del error
- Pruebas de los escenarios de error

Subtareas:

- 5.1 - Mapear los errores de la API (transitorios vs definitivos):
- 5.2 - Implementar backoff:
- 5.3 - Implementar límite de reintentos:
- 5.4 - Emitir `publish.failed` con el detalle del error:
- 5.5 - Pruebas de los escenarios de error:

US-C5 · Cuotas y errores (comprometida, 5 SP)

- 5.1: clasificar los errores en transitorio, diferible (cuota) o definitivo.
- 5.2: política de reintentos configurable por `.env`, y actualizar `.env.example`.
- 5.3: reintento vía reprogramación del job, con la transición `publishing` → `pending` y persistencia de `attempts` y `last_error`.
- 5.4: extender el mock con fallas transitorias controlables.
- 5.5: emitir `publish.failed` al agotar los intentos. Hay que confirmar en la reunión de integración si se emite solo al final.
- 5.6: pruebas de los escenarios de error y del contrato del evento.

Tareas stretch (si hay tiempo)

US-C6 · Máquina de estados de publicación [3 SP · Should]

Como sistema, quiero estados pending → publishing → published/failed, para dar trazabilidad al proceso.

Criterios de aceptación:

- Las transiciones reflejan el resultado real de la API
- GET /publish/{id}/status expone el estado actual

Tareas sugeridas:

- Definir estados y transiciones
- Implementar GET /publish/{id}/status
- Pruebas de la máquina de estados

Subtareas:

- **6.1 - Definir estados y transiciones:**
- **6.2 - Implementar GET /publish/{id}/status:**
- **6.3 - Pruebas de la máquina de estados:**

US-C6 · Máquina de estados (stretch)

- 6.1: pasar todos los cambios de estado por la matriz.
- 6.2: pruebas exhaustivas de transiciones válidas e inválidas.
- 6.3: dejar en el acta que el criterio se demuestra contra el mock.

Actas Reuniones

Acta de <Planning> — Sprint <2>Temas tratados

Campo	Contenido
Fecha y hora	21/09/2026 14:30 - 16:40
Duración	2Hrs
Asistentes	Christian Muños, Nahuel Catrileo, Francisco Ceballos, Hector Chavez
Ausentes justificados	
Facilitador (SM)	Nahuel Catrileo

Se seleccionaron las historias de usuario a desarrollar (US-2 8Sp, US-5 5Sp), historia extra dependiendo de tiempo (US-6 3Sp)

Acta de <Daily > — Sprint <2>Temas tratados

Campo	Contenido
Fecha y hora	24/09/2026 14:30 - 16:40
Duración	15 min
Asistentes	Christian Muños, Nahuel Catrileo, Francisco Ceballos, Hector Chavez
Ausentes justificados	
Facilitador (SM)	Nahuel Catrileo

Historias de usuario divididas en tareas y asignadas al equipo.
Reunión entre Scrum Masters concluye que nuestro módulo debe solucionar problema de JWT del módulo de integración, y con módulo 1 para coordinar formato de metadatos de videos. Posible futura comunicación con módulo 2.

Acta de <Daily > — Sprint <2>Temas tratados

Campo	Contenido
Fecha y hora	28/09/2026 14:30 - 16:40
Duración	15 min
Asistentes	Alonso Chavez, Nahuel Catrileo, Francisco Ceballos, Hector Chavez
Ausentes justificados	
Facilitador (SM)	Nahuel Catrileo

Hector y Nahuel tiene que actualizar backlog y, trello. Asignación clara de tareas para cada integrante, actualización Sprint Backlog para repartición de tareas, reorganización del repositorio.

Acta de <Daily > — Sprint <2>Temas tratados

Campo	Contenido
Fecha y hora	05/10/2026 14:30 - 16:40
Duración	15 min
Asistentes	Alonso Chavez, Nahuel Catrileo, Francisco Ceballos, Hector Chavez, Christian Muñoz
Ausentes justificados	
Facilitador (SM)	Nahuel Catrileo

Christian necesita finalizar pequeños cambios y subir su progreso del US-02 al repositorio. PO Hector término de actualizó Trello y backlog durante semana anterior, fue comunicado al equipo